

Name: Akkshat Shah & Vishwa Patel	Group No: 16
Enrollment number: AU2340246 & AU2340152	Date: 19/11/2025
Final report of the Project	

1) Abstract:

This project is toy Planetary Exploration Rover which is implemented using the A* search to explore different paths and find out the optimal path. The environment is made up of different types of terrains such as rocky, sandy, flat, cliff and hazardous terrains like sandtrap and radiation. There are also charging stations for the rover and start state and the goal state. Every exploration move has a cost which varies according to the terrain type. We implemented the A* search using 4 different types of heuristics - Weighted Manhattan, Euclidean, Terrain-cost Weighted and Battery aware heuristics and the performance and the efficiency is calculated on the basis of explored nodes, total path cost and computation time. A battery aware model is implemented so that it can monitor the energy consumption dynamically and detour to the charging station when battery falls below a certain threshold. After recharging it again resumes its A* search for the goal state. If the battery is not sufficient then we also implemented the self charging system which charges a certain amount of battery which is enough to move to the next exploration state. Lastly, we have tried to implement it visually so that we can clearly state how the agent is performing and in the end of the visualization we have provided the optimal path.

2) Introduction:

Autonomous navigation is a big challenge in the field of robotics, where real time decision making is tough with respect to the environment as there are resource limitations and uncertain terrains. The planetary exploration has to find out the optimal path by reducing the errors, conserving the battery and making decisions based on the environment. Pathfinding methods like A* and greedy best first search rely on heuristic to estimate the cost. Quality of the efficiency and performance depends on the heuristics that you choose. Every heuristic works differently depending on the environment. So testing different heuristics is necessary to compare, and optimize different heuristics.

This project is a toy planetary exploration rover used to navigate the goal on a grid-based terrain. It has realistic constraints such as hazardous terrain, battery consumption and charging stations. Four different heuristics are implemented here and those are: Weighted Manhattan, Euclidean, Terrain-cost Weighted and Battery aware heuristics. These are used to evaluate the performance of the rover to find out the optimal path in the less traversal cost. The simulation also includes advanced features such as dynamic cost according to the terrain, rerouting, battery-aware path finding and self charging.

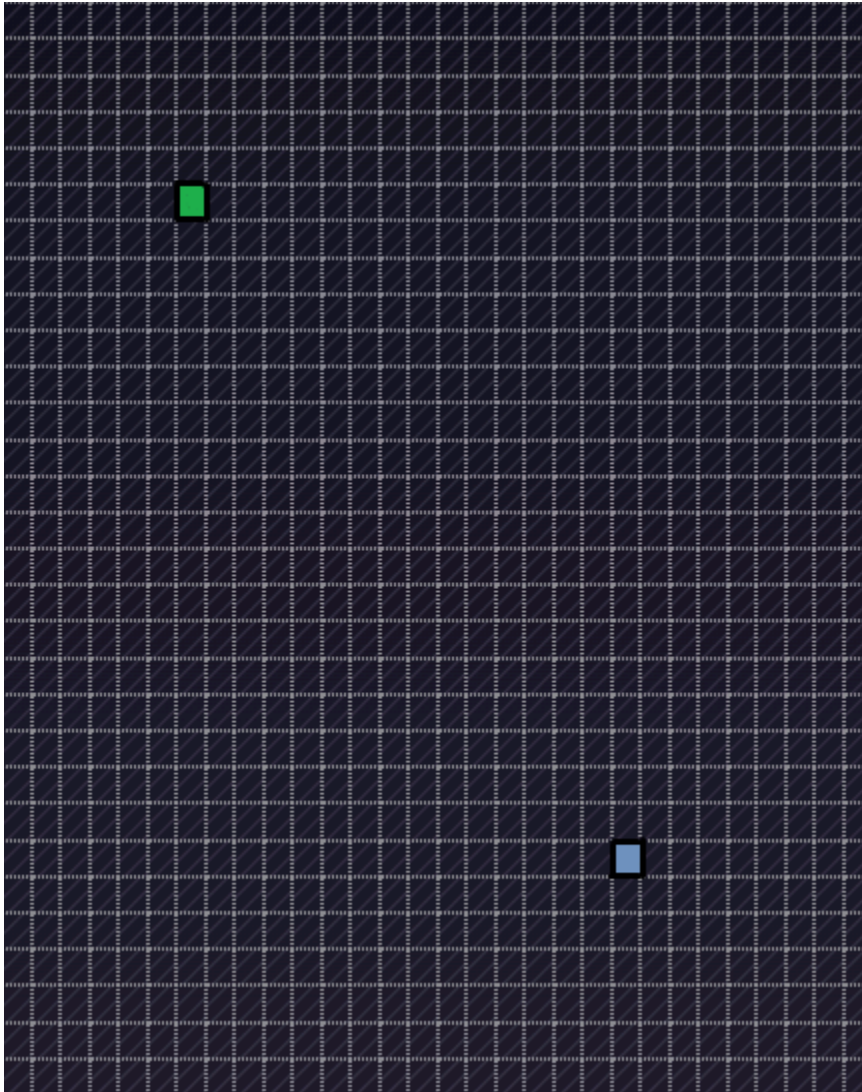
We have run all the heuristics on the same terrain to compare the path length, cost, energy usage and computation time. After completion of all the simulations we have given the optimal paths found by all 4 heuristics by which we can select the most effective heuristic under that particular environment. We have also provided the visualization for the ease of the understanding and to make it look good.

3) Design

Firstly we constructed the grid in which we made different terrains that represent the real life world factors. All the different types of terrains such as flat, sandy, cliff and hazardous terrain such as radiation and sandtrap with the important modules like charging station were randomly placed. Then we fix a cost for each type of terrain across the whole grid in terms of battery consumption. Now for the core design we implemented the A* search from the start state to the goal state in such a way that if battery falls below a certain threshold then it reroute to the charging station, refill its battery and then again resumes its search for the goal state. We then implemented 4 different heuristics to compare each other on the basis of their performance. Each heuristic was implemented in one-by-one order generating separate results. Then we coded the visualization part in which we included the display box which displays which terrain it is exploring, what is the battery percentage, is it in charging mode and if it is charging how much it is charging. All the modules were implemented as independent modules which can make it easier to work on it further and to add new heuristics.

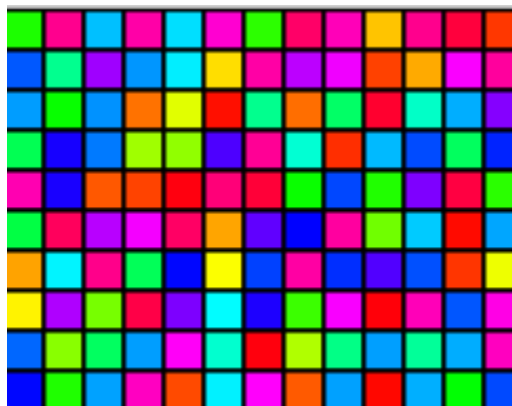
4) Diagrams

→ We selected 30 X 30 grid



This is the 30 X 30 Grid in which the green colour represents the start state and the purple colour represents the goal state. This is how our grid will look like.

→ Grid with environment, it will look similar to the diagram shown below.



5) Method

- 1) **Understanding the problem:** First we went through the whole problem statement and made a list of what we have to implement.
- 2) **Figure out what can be done new:** After understanding we tried to make changes and thought about what other things we can implement.
- 3) **Terrain Setup:** Set up the whole grid based environment where each cell represents a type of terrain.
- 4) **Defining costs:** Then we assigned different costs for different terrains.
- 5) **Defined Heuristics:** Defined and Implemented 4 different heuristics in a way that any of them can be changed later.
- 6) **Implemented A* search:** Then we applied the algorithm of A* to find out the optimal path.
- 7) **Made the rover:** Built a rover that can follow the A* search to find out the optimal path to the goal.
- 8) **Defined restrictions:** Created charging station and battery conditions to make it realistic
- 9) **Backtracking:** Implemented the logic of backtracking when and where required
- 10) **Visualization:** Created a graphical environment using Matplotlib to display the exploration of the rover, terrains and the optimal path.
- 11) **Run program:** Run all the 4 heuristics one by one so that after completion of the program we can compare it.

6) Code of the project

1) Importing Libraries

Python

```
import matplotlib as mpl
mpl.rcParams['animation.embed_limit'] = 100

from agents import Agent
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import matplotlib.gridspec as gridspec
import matplotlib.offsetbox as offsetbox
import heapq
import random
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
from collections import deque
import math
```

2) Defining Terrain cost and colour

Python

```
TERRAIN_COSTS = {
    'flat': 5, 'rocky': 1000, 'sandy': 10, 'radiation': 15,
    'sand_trap': 20, 'cliff': 1000, 'landmark': 5, 'recharge': 5,
}

COLOR_MAP = {
    'flat': '#bfe6b0', 'rocky': '#635341', 'sandy': '#fff9b1',
    'radiation': '#c2fcfc', 'sand_trap': '#ffe53a', 'cliff': '#a9986e',
    'landmark': '#ff2326', 'recharge': '#0385fe', 'rover': '#FF0000',
    'rover_sample': '#FFD700', 'rover_recharge': '#1E90FF',
    'path_trace': '#4863A0', 'explored_node': '#cccccc',
    'start': '#006400', 'goal': '#4B0082', 'final_path_trace': '#8A2BE2'
}
```

3) Sample Grid and heuristics

Python

```
def make_sample_grid(width, height):
    choices = ['flat']*15 + ['rocky']*4 + ['sandy']*6 + ['radiation']*2 +
    ['sand_trap']*2 + ['cliff'] + ['landmark'] + ['recharge']
    return [[random.choice(choices) for _ in range(width)] for _ in
    range(height)]
```

```
def heuristic_weighted_manhattan(a, b, grid=None):
    min_cost = min(TERRAIN_COSTS.values())
    return (abs(a[0]-b[0]) + abs(a[1]-b[1])) * min_cost
```

```
def heuristic_euclidean(a, b, grid=None):
    min_cost = min(TERRAIN_COSTS.values())
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2) * min_cost
```

```
def heuristic_terrain_weighted(a, b, grid):
    x0, y0 = a
    x1, y1 = b
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    steps = max(dx, dy)
    if steps == 0:
        return 0
    total_cost = 0
    for i in range(steps + 1):
        x = int(round(x0 + (x1 - x0) * i / steps))
        y = int(round(y0 + (y1 - y0) * i / steps))
        terrain = grid[y][x]
        total_cost += TERRAIN_COSTS.get(terrain, 1000)
    avg_cost = total_cost / (steps + 1)
    euclid_dist = math.sqrt(dx * dx + dy * dy)
    return euclid_dist * avg_cost
```

```
def heuristic_battery_aware(a, b, grid, rover_battery=100):
    dist = heuristic_terrain_weighted(a, b, grid)
    penalty = 0
    if rover_battery < 30:
        penalty = (30 - rover_battery) * 10
```

```
return dist + penalty
```

4) A* search

Python

```
def astar_search(grid, start, goal, heuristic_func, rover_battery=100):
    width, height = len(grid[0]), len(grid)
    open_set = []
    heapq.heappush(open_set, (0, start))
    came_from = {}
    g_score = {start: 0}
    visited_order = []
    while open_set:
        _, current = heapq.heappop(open_set)
        visited_order.append(current)
        if current == goal:
            path = []
            total_cost = g_score[goal]
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path, total_cost, visited_order
        x, y = current
        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < width and 0 <= ny < height:
                terrain = grid[ny][nx]
                cost = TERRAIN_COSTS.get(terrain, 1000)
                if cost >= 1000:
                    continue # block impassable terrains such as cliff or
                    rocky

                tentative_g = g_score[current] + cost
                if (nx, ny) not in g_score or tentative_g < g_score[(nx, ny)]:
                    came_from[(nx, ny)] = current
                    g_score[(nx, ny)] = tentative_g
                if heuristic_func == heuristic_battery_aware:
                    h = heuristic_func((nx, ny), goal, grid, rover_battery)
                elif heuristic_func == heuristic_weighted_manhattan:
                    h = heuristic_func((nx, ny), goal)
                else:
                    h = heuristic_func((nx, ny), goal, grid)
```

```

        f = tentative_g + h
        heapq.heappush(open_set, (f, (nx, ny)))
    return None, float('inf'), visited_order

```

5) Recharge within

Python

```

def find_recharge_within(grid, start, max_distance=2):
    width, height = len(grid[0]), len(grid)
    queue = deque([(start, 0)])
    visited = set([start])
    while queue:
        (x, y), dist = queue.popleft()
        if grid[y][x] == 'recharge' and dist > 0:
            return (x, y)
        if dist < max_distance:
            for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nx, ny = x + dx, y + dy
                if 0 <= nx < width and 0 <= ny < height:
                    terrain = grid[ny][nx]
                    if terrain not in ['rocky', 'cliff'] and (nx, ny) not in
visited:
                        visited.add((nx, ny))
                        queue.append(((nx, ny), dist + 1))
    return None

```

6) random start and goal

Python

```

def generate_valid_start_goal(width, height, min_delta=15):
    blocked = ['rocky', 'cliff', 'recharge'] # hazardous terrains allowed
    since not blocking
    while True:
        start_x = random.randint(0, width - min_delta - 1)
        start_y = random.randint(0, height - min_delta - 1)
        if terrain_grid[start_y][start_x] in blocked:
            continue
        goal_x = random.randint(start_x + min_delta, width - 1)
        goal_y = random.randint(start_y + min_delta, height - 1)

```



```
if terrain_grid[goal_y][goal_x] in blocked:
    continue
return (start_x, start_y), (goal_x, goal_y)
```

7) Agent

Python

```
class RoverAgent(Agent):
    def __init__(self):
        super().__init__(program=self.program)
        self.location = (0, 0)
        self.path = []
        self.path_index = 0
        self.battery = 100
        self.original_goal = None
        self.recharge_goal = None
        self.on_recharge_detour = False

    def set_path(self, path):
        self.path = path
        self.path_index = 0

    def program(self, percept):
        if self.path and self.path_index < len(self.path):
            next_loc = self.path[self.path_index]
            self.path_index += 1
            return ('move_to', next_loc)
        return None
```

8) Environment

Python

```
class RoverEnvironment:
    def __init__(self, grid, start, goal):
        self.grid = grid
        self.start = start
        self.goal = goal
        self.width = len(grid[0])
```

```

self.height = len(grid)
self.rover = None
self.patches = {}
self.borders = []

self.fig = plt.figure(figsize=(18, 12))
gs = gridspec.GridSpec(1, 2, width_ratios=[4, 1])
self.ax = self.fig.add_subplot(gs[0])
self.side_ax = self.fig.add_subplot(gs[1])
self.side_ax.axis('off')
self.ax.set_title("Planetary Exploration Rover")

self.ax.set_xlim(0, self.width)
self.ax.set_ylim(0, self.height)
self.ax.set_aspect('equal')
self.ax.invert_yaxis()
self.ax.set_xticks(range(self.width))
self.ax.set_yticks(range(self.height))
self.ax.grid(True, which='major', color='gray', linestyle='-',
linewidth=0.3)

for y in range(self.height):
    for x in range(self.width):
        terrain = self.grid[y][x]
        color = COLOR_MAP.get(terrain, "#ffffff")
        rect = patches.Rectangle((x, y), 1, 1, linewidth=0.2,
edgecolor='gray', facecolor=color, zorder=1)
        self.ax.add_patch(rect)
        self.patches[(x, y)] = rect

self.patches[start].set_facecolor(COLOR_MAP['start'])
self.patches[goal].set_facecolor(COLOR_MAP['goal'])
border_width = 2
start_border = patches.Rectangle(start, 1, 1, linewidth=border_width,
edgecolor='black', facecolor='none', zorder=5)
goal_border = patches.Rectangle(goal, 1, 1, linewidth=border_width,
edgecolor='black', facecolor='none', zorder=5)
self.ax.add_patch(start_border)
self.ax.add_patch(goal_border)
self.borders.extend([start_border, goal_border])

self.rover_patch = patches.Rectangle((start[0], start[1]), 1, 1,
linewidth=2, edgecolor='black', facecolor=COLOR_MAP['rover'], zorder=10)
self.ax.add_patch(self.rover_patch)

```

```

self.info_box = offsetbox.AnchoredText(
    "", loc='upper center', pad=0.7, borderpad=1, frameon=True,
    bbox_to_anchor=(0.5, 0.18), bbox_transform=self.side_ax.transAxes,
    prop=dict(size=14, family='monospace')
)
self.side_ax.add_artist(self.info_box)

legend_patches = [
    patches.Patch(color=COLOR_MAP['flat'], label='Flat'),
    patches.Patch(color=COLOR_MAP['rocky'], label='Rocky'),
    patches.Patch(color=COLOR_MAP['sandy'], label='Sandy'),
    patches.Patch(color=COLOR_MAP['radiation'], label='Radiation'),
    patches.Patch(color=COLOR_MAP['sand_trap'], label='Sand Trap'),
    patches.Patch(color=COLOR_MAP['cliff'], label='Cliff'),
    patches.Patch(color=COLOR_MAP['landmark'], label='Landmark'),
    patches.Patch(color=COLOR_MAP['recharge'], label='Recharge'),
    patches.Patch(color=COLOR_MAP['start'], label='Start (Dark
Green)'),
    patches.Patch(color=COLOR_MAP['goal'], label='Goal (Dark Purple)'),
]
self.legend_box = self.side_ax.legend(handles=legend_patches,
                                     loc='upper center',
bbox_to_anchor=(0.5, 0.56),
                                     frameon=True, fancybox=True,
borderpad=1,
                                     fontsize=13, title='Terrain
Legend', title_fontsize=14)
self.legend_box.get_frame().set_edgecolor('black')
self.legend_box.get_frame().set_linewidth(1.8)
self.info_box.set_zorder(self.legend_box.get_frame().get_zorder() + 1)

def add_rover(self, rover, location):
    self.rover = rover
    self.rover.location = location
    self.rover_patch.set_xy(location)

def update_rover(self, color, visited, explored, path_trace,
show_info=True, charge_status=False, charge_amt=0):
    x, y = self.rover.location
    self.rover_patch.set_xy((x, y))
    self.rover_patch.set_facecolor(color)
    for loc, patch in self.patches.items():
        terrain = self.grid[loc[1]][loc[0]]

```

```

        patch.set_facecolor(COLOR_MAP.get(terrain, "#ffffff"))
    self.patches[self.start].set_facecolor(COLOR_MAP['start'])
    self.patches[self.goal].set_facecolor(COLOR_MAP['goal'])
    for border in self.borders:
        border.set_visible(True)
    for loc in explored:
        if loc not in path_trace and loc != self.start and loc !=
self.goal:
            self.patches[loc].set_facecolor(COLOR_MAP['explored_node'])
    for loc in path_trace:
        if loc != (x, y) and loc != self.start and loc != self.goal:
            self.patches[loc].set_facecolor(COLOR_MAP['path_trace'])
    if show_info:
        terrain_name = self.grid[y][x].capitalize()
        charging_str = 'Yes' if charge_status else 'No'
        info = (
            f"Battery: {self.rover.battery}%\n"
            f"Terrain: {terrain_name}\n"
            f"Charging: {charging_str}\n"
            f"Charge Amount: {charge_amt}%"
        )
        self.info_box.txt.set_text(info)

def show_full_path(self, path, final=False):
    color_key = 'final_path_trace' if final else 'path_trace'
    for loc in path:
        if loc != self.start and loc != self.goal:
            self.patches[loc].set_facecolor(COLOR_MAP[color_key])
    self.rover.location = path[-1]
    self.rover_patch.set_xy(self.rover.location)
    self.rover_patch.set_facecolor(COLOR_MAP['rover'])
    self.update_rover(COLOR_MAP['rover'], set(), set(), set(path),
show_info=True)

```

9) Simulation

Python

```

def run_simulation_with_heuristic(grid, heuristic_func, heuristic_name, start,
goal, rover_battery=100):
    print(f"Heuristic: {heuristic_name} | Start: {start} | Goal: {goal}")

```

```

    path, total_cost, visited_order = astar_search(grid, start, goal,
    heuristic_func, rover_battery=rover_battery)
    if path:
        print(f"Total path cost: {total_cost}")
        print(f"Final path length: {len(path)}")
        print(f"Final path: {path}")
    else:
        print("No valid path found between start and goal.")

    env = RoverEnvironment(grid, start, goal)
    rover = RoverAgent()
    rover.original_goal = goal
    rover.recharge_goal = None
    rover.on_recharge_detour = False
    rover.battery = rover_battery
    rover.set_path(path if path else [])
    env.add_rover(rover, start)

    visited = set()
    explored = set()
    path_trace = set()
    backtracking_stack = []

    total_exploration_frames = len(visited_order)
    total_path_frames = len(path) if path else 0
    showing_final_path = False

    def find_last_safe_cell(traced_cells):
        for loc in reversed(traced_cells):
            x, y = loc
            if grid[y][x] not in ['sand_trap', 'radiation']:
                return loc
        return None

    def update(frame):
        nonlocal showing_final_path, total_path_frames, backtracking_stack
        total_frames = total_exploration_frames + total_path_frames

        if backtracking_stack:
            next_cell = backtracking_stack[-1]
            x_next, y_next = next_cell
            cost = TERRAIN_COSTS.get(grid[y_next][x_next], 5)
            if rover.battery < cost:
                charge_amt = min(5, 100 - rover.battery)

```

```

        old_battery = rover.battery
        rover.battery = min(rover.battery + 5, 100)
        charge_status = True
        print(f"🔋 Battery low during backtracking at {rover.location}:
charging {old_battery}% → {rover.battery}%")
        env.update_rover(COLOR_MAP['rover'], visited, explored,
path_trace, charge_status=charge_status, charge_amt=charge_amt)
        return [env.rover_patch]
        rover.location = backtracking_stack.pop()
        x, y = rover.location
        before = rover.battery
        if rover.battery >= cost:
            rover.battery -= cost
            rover.battery = max(rover.battery, 0)
        env.update_rover(COLOR_MAP['rover'], visited, explored, path_trace,
charge_status=False, charge_amt=0)
        print(f"🔄 Backtracking to {rover.location} | Battery: {before}% →
{rover.battery}%")
        return [env.rover_patch]

    if frame < total_exploration_frames:
        current_node = visited_order[frame]
        x, y = current_node
        terrain = grid[y][x]

        if terrain in ['sand_trap', 'radiation']:
            print(f"⚠️ Hazardous terrain at exploration node
{current_node}. Backtracking.")
            safe_cell = find_last_safe_cell(list(visited))
            if safe_cell:
                backtracking_stack.clear()
                loc_list = list(visited)
                for loc in reversed(loc_list):
                    backtracking_stack.append(loc)
                    if loc == safe_cell:
                        break
                print(f"Backtracking {len(backtracking_stack)} steps to
safe cell {safe_cell} during exploration.")
                for loc in backtracking_stack:
                    if loc in visited:
                        visited.remove(loc)
                return [env.rover_patch]
            else:

```

```

        print("No safe cell to backtrack to during exploration.
Stopping.")

        return [env.rover_patch]

    before = rover.battery
    cost_display = TERRAIN_COSTS.get(terrain, 5)
    cost = TERRAIN_COSTS.get(terrain, 1000)
    color = COLOR_MAP['rover']
    charge_status, charge_amt = False, 0
    msg = ""

    if rover.battery < cost and terrain != 'recharge':
        charge_amt = min(5, 100 - rover.battery)
        old_battery = rover.battery
        rover.battery = min(rover.battery + 5, 100)
        charge_status = True
        msg = f"⚡ Not enough battery ({old_battery}%) for exploration
move {current_node} (needs {cost_display}%). Self-charging to {rover.battery}%"
        print(msg)
        env.update_rover(color, visited, explored, path_trace,
charge_status=charge_status, charge_amt=charge_amt)
        return [env.rover_patch]

    if 20 <= rover.battery <= 25 and not rover.on_recharge_detour and
terrain != 'recharge':
        recharge_loc = find_recharge_within(grid, rover.location,
max_distance=2)
        if recharge_loc:
            print(f"Detour: Battery between 20-25%. Recharge station
found at {recharge_loc}")
            rover.recharge_goal = recharge_loc
            if not rover.original_goal:
                rover.original_goal = rover.path[-1] if rover.path else
None

            path_to_recharge, _, _ = astar_search(grid, rover.location,
recharge_loc, heuristic_weighted_manhattan)
            if path_to_recharge:
                rover.set_path(path_to_recharge)
                rover.on_recharge_detour = True
                total_path_frames = len(rover.path)
                return [env.rover_patch]

    rover.battery -= cost
    rover.battery = max(rover.battery, 0)

```

```

        if terrain == 'landmark':
            color = COLOR_MAP['rover_sample']
            msg = f"🔬 Sample collected at {current_node}! Battery:
{before}% → {rover.battery}%"
        elif terrain == 'recharge':
            color = COLOR_MAP['rover_recharge']
            charge_amt = 100 - rover.battery
            if rover.battery < 100:
                msg = f"🔋 Recharge station at {current_node}! Battery
before: {before}% → 100%"
                charge_status = True
                rover.battery = 100
                if rover.on_recharge_detour:
                    rover.on_recharge_detour = False
                    rover.recharge_goal = None
                    if rover.original_goal:
                        path_back, _, _ = astar_search(grid, rover.location,
rover.original_goal, heuristic_weighted_manhattan)
                        if path_back:
                            rover.set_path(path_back)
                            total_path_frames = len(rover.path)
                else:
                    msg = f"Exploring node {current_node} | Terrain: {terrain} |
Battery: {before}% → {rover.battery}%"

            print(msg)
            visited.add(current_node)
            explored.add(current_node)
            rover.location = current_node
            env.update_rover(color, visited, explored, path_trace,
charge_status=charge_status, charge_amt=charge_amt)
            return [env.rover_patch]

    path_frame = frame - total_exploration_frames

    if path_frame >= len(rover.path):
        showing_final_path = True
        env.show_full_path(path, final=True)
        return [env.rover_patch]

    rover.location = rover.path[path_frame]
    path_trace.add(rover.location)
    x, y = rover.location
    terrain = grid[y][x]

```



```

before = rover.battery
cost_display = TERRAIN_COSTS.get(terrain, 5)
cost = TERRAIN_COSTS.get(terrain, 1000)
color = COLOR_MAP['rover']
charge_status, charge_amt = False, 0
msg = ""

if rover.battery < cost and terrain != 'recharge':
    charge_amt = min(5, 100 - rover.battery)
    old_battery = rover.battery
    rover.battery = min(rover.battery + 5, 100)
    charge_status = True
    msg = f"⚡ Not enough battery ({old_battery}%) for move
{rover.location} (needs {cost_display}%). Self-charging to {rover.battery}%"
    print(msg)
    env.update_rover(color, visited, explored, path_trace,
charge_status=charge_status, charge_amt=charge_amt)
    return [env.rover_patch]

if 20 <= rover.battery <= 25 and not rover.on_recharge_detour and
terrain != 'recharge':
    recharge_loc = find_recharge_within(grid, rover.location,
max_distance=2)
    if recharge_loc:
        print(f"Detour: Battery between 20-25%. Recharge station found
at {recharge_loc}")
        rover.recharge_goal = recharge_loc
        if not rover.original_goal:
            rover.original_goal = rover.path[-1] if rover.path else
None
        path_to_recharge, _, _ = astar_search(grid, rover.location,
recharge_loc, heuristic_weighted_manhattan)
        if path_to_recharge:
            rover.set_path(path_to_recharge)
            rover.on_recharge_detour = True
            total_path_frames = len(rover.path)
            return [env.rover_patch]

if terrain in ['sand_trap', 'radiation']:
    print(f"⚠ Hazardous terrain at {rover.location} during path
follow.")
    safe_cell = find_last_safe_cell(list(path_trace))
    if safe_cell:
        backtracking_stack.clear()

```

```

        loc_list = list(path_trace)
        for loc in reversed(loc_list):
            backtracking_stack.append(loc)
            if loc == safe_cell:
                break
            print(f"Backtracking {len(backtracking_stack)} steps to safe
cell {safe_cell} during path follow.")
            for loc in backtracking_stack:
                if loc in path_trace:
                    path_trace.remove(loc)
            return [env.rover_patch]
    else:
        print("No safe cell to backtrack to during path follow. Rover
halted.")

        return [env.rover_patch]

    rover.battery -= cost
    rover.battery = max(rover.battery, 0)
    if terrain == 'landmark':
        msg = f"🔬 Sample collected at {rover.location}! Battery: {before}%
→ {rover.battery}%"
    elif terrain == 'recharge':
        color = COLOR_MAP['rover_recharge']
        charge_amt = 100 - rover.battery
        if rover.battery < 100:
            msg = f"🔌 Recharge station at {rover.location}! Battery
before: {before}% → 100%"
            charge_status = True
            rover.battery = 100
            if rover.on_recharge_detour:
                rover.on_recharge_detour = False
                rover.recharge_goal = None
                if rover.original_goal:
                    path_back, _, _ = astar_search(grid, rover.location,
rover.original_goal, heuristic_weighted_manhattan)
                    if path_back:
                        rover.set_path(path_back)
                        total_path_frames = len(rover.path)
            else:
                msg = f"Rover moved to {rover.location} | Terrain: {terrain} |
Battery: {before}% → {rover.battery}%"

        if rover.location == env.goal:

```

```

        msg = f"✅ Rover reached the goal {env.goal}! Final battery:
{rover.battery}%"

    if msg:
        print(msg)
        env.update_rover(color, visited, explored, path_trace,
            charge_status=charge_status, charge_amt=charge_amt)
        return [env.rover_patch]

    ani = FuncAnimation(env.fig, update, frames=total_exploration_frames +
        total_path_frames if total_exploration_frames + total_path_frames > 0 else 1,
        interval=80, blit=True, repeat=False)
    display(HTML(ani.to_jshtml()))

```

10) seed value to generate

```

Python
random.seed()
WIDTH, HEIGHT = 30, 30
terrain_grid = make_sample_grid(WIDTH, HEIGHT)

start, goal = generate_valid_start_goal(WIDTH, HEIGHT, min_delta=15)
print(f"Common Start: {start} (terrain: {terrain_grid[start[1]][start[0]]}) |
Common Goal: {goal} (terrain: {terrain_grid[goal[1]][goal[0]]})")

heuristics = [
    ("Manhattan Distance", heuristic_weighted_manhattan),
    ("Euclidean Distance", heuristic_euclidean),
    ("Terrain-cost Weighted", heuristic_terrain_weighted),
    ("Battery-aware", heuristic_battery_aware)
]

for name, heuristic_func in heuristics:
    run_simulation_with_heuristic(terrain_grid, heuristic_func, name, start,
        goal, rover_battery=100)

```

7) Comparison of the 4 heuristics:

1. *Weighted Manhattan Distance Heuristic*

Formula:

$$h(a, b) = (|x_a - x_b| + |y_a - y_b|) \times c_{\min}$$

This heuristic multiplies the standard Manhattan distance (sum of horizontal and vertical moves) by the minimum terrain cost in the grid. It assumes movement is restricted to four cardinal directions. Because it uses the least possible movement cost, it provides an admissible (non-overestimating) estimate.

Characteristics:

- Admissible and consistent (guarantees optimality).
- Performs well in uniform cost terrains such as flat or evenly mixed maps.
- Underestimates in varied or obstacle-heavy maps since it doesn't factor terrain costs beyond the minimum cost.

Observations in Simulation:

- The rover explores more nodes.
- Reliable but slower in complex terrains because it cannot anticipate difficult zones.

2. *Euclidean Distance Heuristic*

Formula:

$$h(a, b) = \text{sqrt}((x_a - x_b)^2 + (y_a - y_b)^2) \times c_{\min}$$

This heuristic uses the straight-line distance multiplied by the minimum terrain cost. It estimates the shortest physical path regardless of grid direction constraints.

Characteristics:

- Slightly more optimistic than Manhattan distance.
- Can better approximate true travel cost when diagonal proximity matters, though the rover moves only in four directions.
- Still admissible, but not always ideal for grid-based motion without diagonal movement.

Observations in Simulation:

- Similar exploration count to Manhattan but sometimes shorter search times.
- Generates smoother paths through open spaces.

3. *Terrain-Cost Weighted Heuristic*

Formula:

$$h(a, b) = \text{EuclideanDistance}(a, b) \times c_avg_path$$

This heuristic considers geometric distance but also evaluates the expected average terrain difficulty between the current node and the goal. Since it incorporates terrain cost dynamically, it adapts effectively to rough or risky paths.

Characteristics:

- More informed, reducing unnecessary exploration near costly terrains.
- Non-admissible (can overestimate due to terrain weighting).
- Performs best when terrains vary significantly.

Observations in Simulation:

- Produces shorter path lengths with fewer detours.
- Avoids expensive terrains like rocky or hazardous zones.
- Improves travel efficiency but may miss the theoretically optimal path.

4. *Battery-Aware Heuristic*

Formula:

$h(a, b) = \text{TerrainWeightedHeuristic}(a, b) + \text{BatteryPenalty}$
(the penalty increases when battery percentage is below 30%)

This heuristic extends the terrain-weighted approach by adding a penalty that increases as battery levels drop. This makes the algorithm prioritize shorter or safer routes or detours toward recharge stations.

Characteristics:

- Context-sensitive; incorporates rover battery state.
- Encourages adaptive decision-making and efficient recharging.
- Non-admissible, but highly realistic for real rover missions.

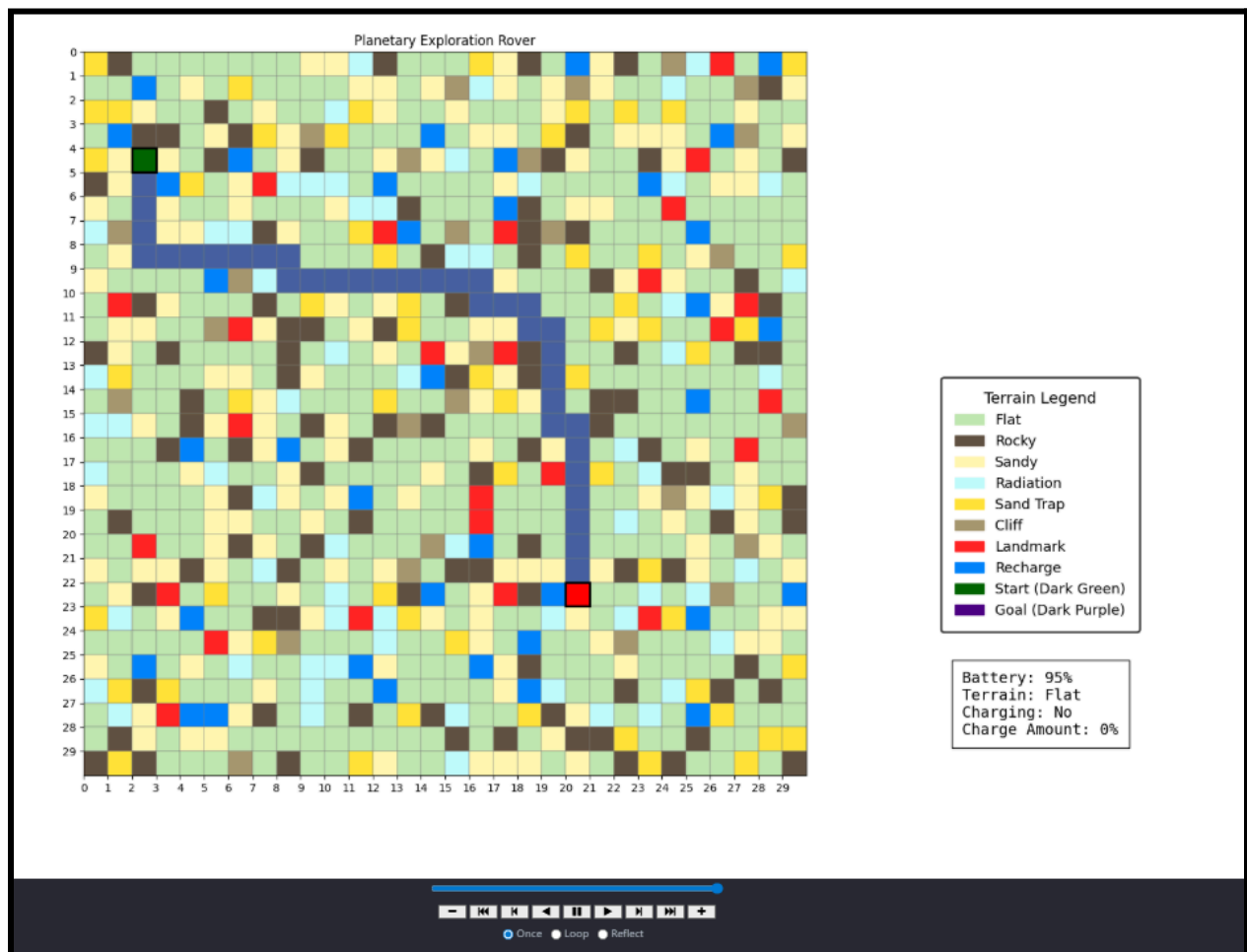
Observations in Simulation:

- Automatically redirects toward recharge stations when energy is low.
- Balances path cost with battery efficiency.
- May increase runtime but improves mission success rate.

8) Screenshots of the program execution

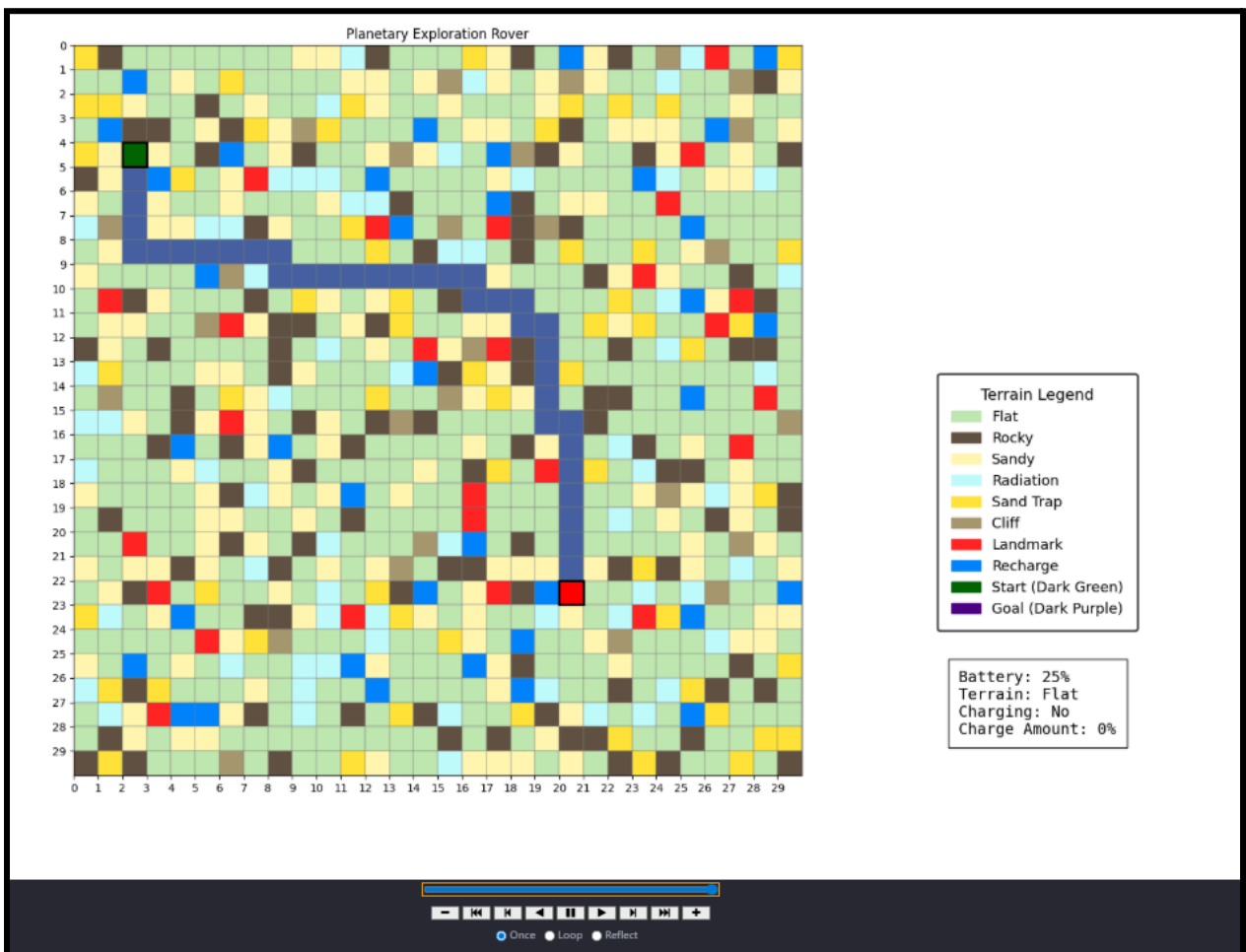
1) Manhattan distance

```
Common Start: (2, 4) (terrain: flat) | Common Goal: (20, 22) (terrain: flat)
Heuristic: Manhattan Distance | Start: (2, 4) | Goal: (20, 22)
Total path cost: 195
Final path length: 37
Final path: [(2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (3, 8), (4, 8), (5, 8), (6, 8), (7, 8), (8, 8), (8, 9), (9, 9), (10, 9), (11, 9),
Exploring node (2, 4) | Terrain: flat | Battery: 100% → 95%
Exploring node (2, 4) | Terrain: flat | Battery: 95% → 90%
Exploring node (2, 4) | Terrain: flat | Battery: 90% → 85%
Exploring node (2, 4) | Terrain: flat | Battery: 85% → 80%
  Recharge station at (2, 5)! Battery before: 80% → 100%
Exploring node (2, 6) | Terrain: flat | Battery: 100% → 95%
  Recharge station at (3, 5)! Battery before: 95% → 100%
Exploring node (2, 7) | Terrain: sandy | Battery: 100% → 90%
Exploring node (2, 8) | Terrain: flat | Battery: 90% → 85%
Exploring node (2, 9) | Terrain: flat | Battery: 85% → 80%
Exploring node (3, 4) | Terrain: sandy | Battery: 80% → 70%
Exploring node (3, 6) | Terrain: sandy | Battery: 70% → 60%
Exploring node (3, 8) | Terrain: flat | Battery: 60% → 55%
Exploring node (3, 9) | Terrain: flat | Battery: 55% → 50%
Exploring node (4, 4) | Terrain: flat | Battery: 50% → 45%
Exploring node (4, 6) | Terrain: flat | Battery: 45% → 40%
Exploring node (4, 8) | Terrain: flat | Battery: 40% → 35%
Exploring node (4, 9) | Terrain: flat | Battery: 35% → 30%
Exploring node (4, 10) | Terrain: flat | Battery: 30% → 25%
Detour: Battery between 20-25%. Recharge station found at (5, 9)
Exploring node (4, 12) | Terrain: flat | Battery: 25% → 20%
Exploring node (4, 13) | Terrain: flat | Battery: 20% → 15%
Exploring node (5, 6) | Terrain: flat | Battery: 15% → 10%
Exploring node (5, 8) | Terrain: flat | Battery: 10% → 5%
```



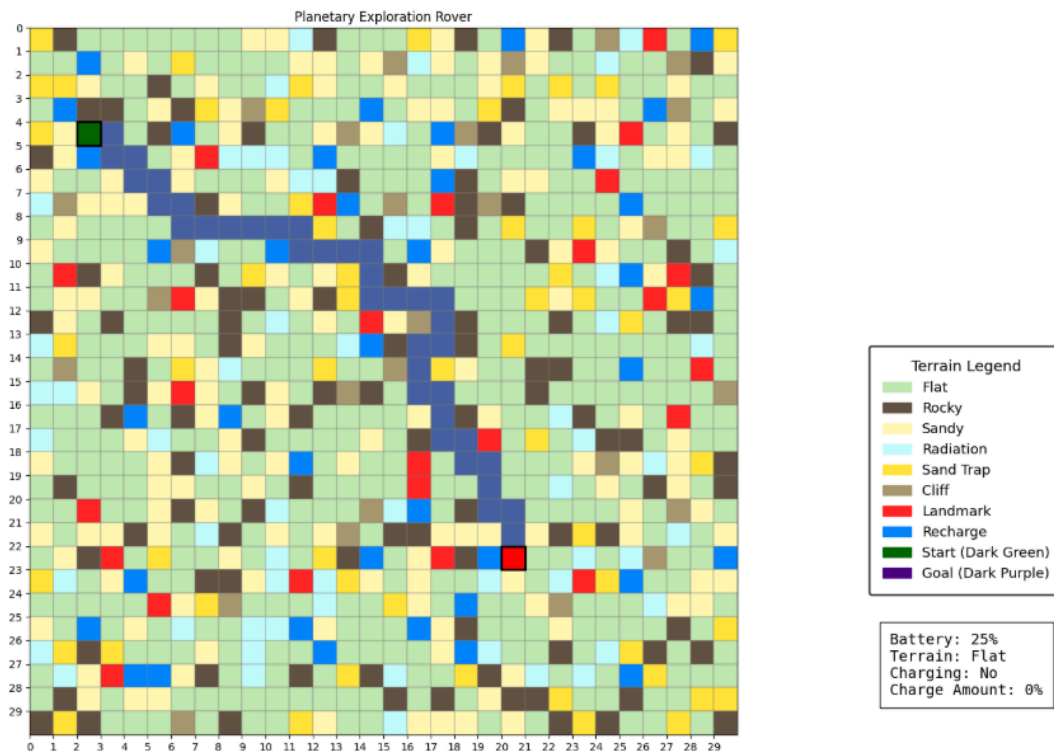
2) Euclidean Distance

```
Heuristic: Euclidean Distance | Start: (2, 4) | Goal: (20, 22)
Total path cost: 195
Final path length: 37
Final path: [(2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (3, 8), (4, 8), (5, 8), (6, 8), (7, 8), (8, 8), (8, 9)]
Exploring node (2, 4) | Terrain: flat | Battery: 100% → 95%
Exploring node (2, 4) | Terrain: flat | Battery: 95% → 90%
Exploring node (2, 4) | Terrain: flat | Battery: 90% → 85%
Exploring node (2, 4) | Terrain: flat | Battery: 85% → 80%
  Recharge station at (2, 5)! Battery before: 80% → 100%
  Recharge station at (3, 5)! Battery before: 100% → 100%
Exploring node (2, 6) | Terrain: flat | Battery: 100% → 95%
Exploring node (3, 4) | Terrain: sandy | Battery: 95% → 85%
Exploring node (4, 4) | Terrain: flat | Battery: 85% → 80%
Exploring node (3, 6) | Terrain: sandy | Battery: 80% → 70%
Exploring node (2, 7) | Terrain: sandy | Battery: 70% → 60%
Exploring node (4, 6) | Terrain: flat | Battery: 60% → 55%
Exploring node (2, 8) | Terrain: flat | Battery: 55% → 50%
Exploring node (1, 6) | Terrain: flat | Battery: 50% → 45%
Exploring node (5, 6) | Terrain: flat | Battery: 45% → 40%
Exploring node (3, 8) | Terrain: flat | Battery: 40% → 35%
Exploring node (1, 4) | Terrain: sandy | Battery: 35% → 25%
Detour: Battery between 20-25%. Recharge station found at (1, 3)
Exploring node (4, 8) | Terrain: flat | Battery: 25% → 20%
Exploring node (3, 9) | Terrain: flat | Battery: 20% → 15%
Exploring node (1, 5) | Terrain: sandy | Battery: 15% → 5%
Exploring node (5, 8) | Terrain: flat | Battery: 5% → 0%
  Not enough battery (0%) for exploration move (4, 9) (needs 5%). Self-charging to 5%
  Not enough battery (5%) for exploration move (3, 7) (needs 10%). Self-charging to 10%
Exploring node (6, 8) | Terrain: flat | Battery: 10% → 5%
```



3) Terrain Cost weighted

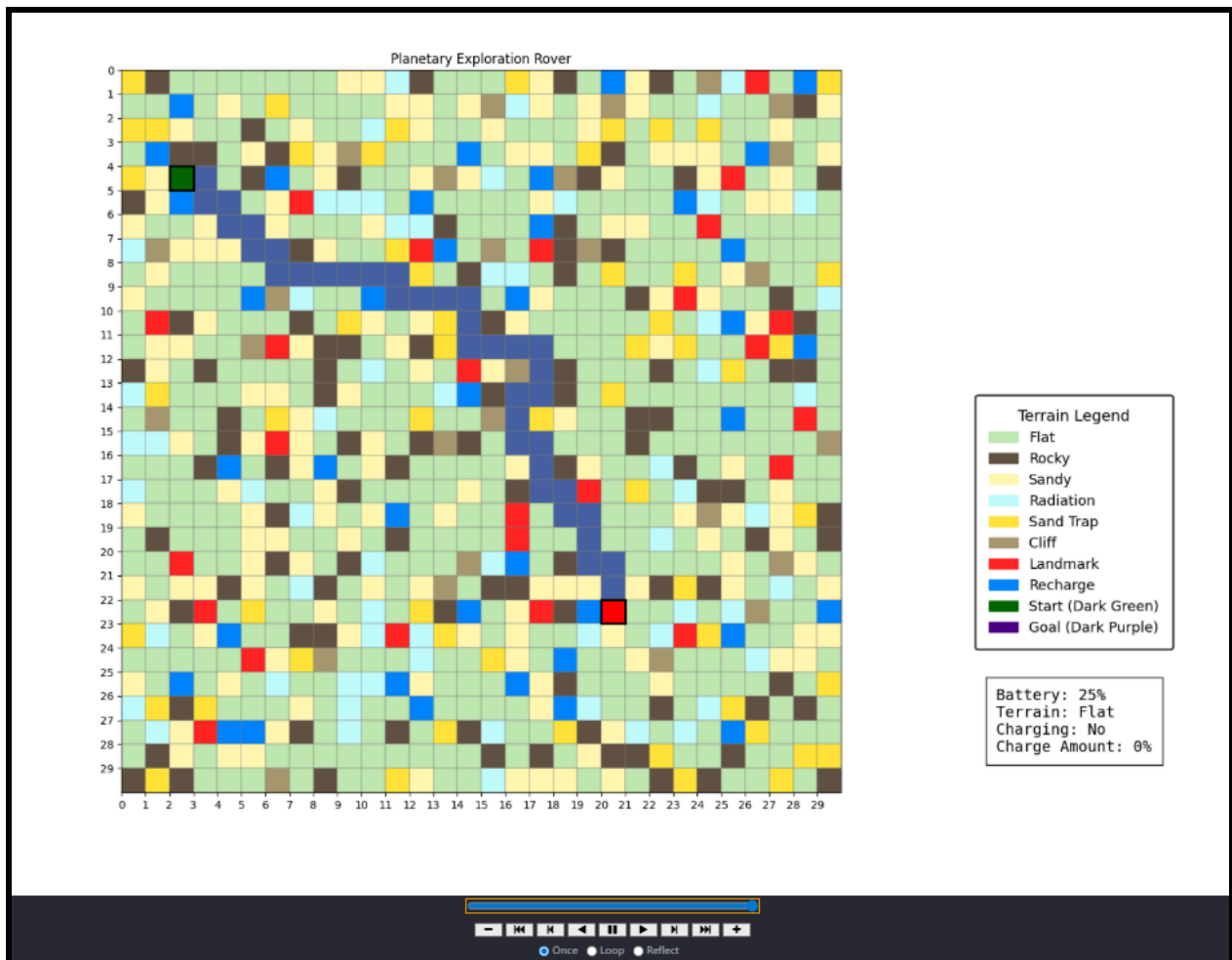
```
Heuristic: Terrain-cost Weighted | Start: (2, 4) | Goal: (20, 22)
Total path cost: 285
Final path length: 39
Final path: [(2, 4), (3, 4), (3, 5), (4, 5), (4, 6), (5, 6), (5, 7), (6, 7), (6, 8), (7, 8), (8, 8), (9, 8), (10, 8), (11, 8), (11, 9), (10, 9), (9, 9), (8, 9), (7, 9), (6, 9), (5, 9), (4, 9), (3, 9), (3, 8), (2, 8), (2, 7), (2, 6), (2, 5), (2, 4)]
Exploring node (2, 4) | Terrain: flat | Battery: 100% → 95%
Exploring node (2, 4) | Terrain: flat | Battery: 95% → 90%
Exploring node (2, 4) | Terrain: flat | Battery: 90% → 85%
Exploring node (2, 4) | Terrain: flat | Battery: 85% → 80%
Exploring node (3, 4) | Terrain: sandy | Battery: 80% → 70%
    🟢 Recharge station at (3, 5)! Battery before: 70% → 100%
    ⚠️ Hazardous terrain at exploration node (4, 5). Backtracking.
    Backtracking 1 steps to safe cell (3, 5) during exploration.
    📦 Backtracking to (3, 5) | Battery: 100% → 95%
    Exploring node (5, 6) | Terrain: flat | Battery: 95% → 90%
    ⚠️ Hazardous terrain at exploration node (5, 7). Backtracking.
    Backtracking 1 steps to safe cell (3, 4) during exploration.
    📦 Backtracking to (3, 4) | Battery: 90% → 80%
    Exploring node (6, 8) | Terrain: flat | Battery: 80% → 75%
    Exploring node (7, 8) | Terrain: flat | Battery: 75% → 70%
    Exploring node (8, 8) | Terrain: flat | Battery: 70% → 65%
    Exploring node (8, 9) | Terrain: flat | Battery: 65% → 60%
    Exploring node (9, 8) | Terrain: flat | Battery: 60% → 55%
    Exploring node (10, 8) | Terrain: flat | Battery: 55% → 50%
    Exploring node (11, 8) | Terrain: flat | Battery: 50% → 45%
    Exploring node (11, 9) | Terrain: flat | Battery: 45% → 40%
    ...
    Rover moved to (20, 16) | Terrain: flat | Battery: 40% → 35%
    Rover moved to (20, 17) | Terrain: flat | Battery: 35% → 30%
    Rover moved to (20, 18) | Terrain: flat | Battery: 30% → 25%
    Detour: Battery between 20-25%. Recharge station found at (20, 21)
    Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



4) Battery- aware

```
Heuristic: Battery-aware | Start: (2, 4) | Goal: (20, 22)
Total path cost: 285
Final path length: 39
Final path: [(2, 4), (3, 4), (3, 5), (4, 5), (4, 6), (5, 6), (5, 7), (6, 7), (6, 8), (7, 8), (8, 8), (9, 8)]
Exploring node (2, 4) | Terrain: flat | Battery: 100% → 95%
Exploring node (2, 4) | Terrain: flat | Battery: 95% → 90%
Exploring node (2, 4) | Terrain: flat | Battery: 90% → 85%
Exploring node (2, 4) | Terrain: flat | Battery: 85% → 80%
Exploring node (3, 4) | Terrain: sandy | Battery: 80% → 70%
  🟢 Recharge station at (3, 5)! Battery before: 70% → 100%
  ⚠️ Hazardous terrain at exploration node (4, 5). Backtracking.
  Backtracking 1 steps to safe cell (3, 5) during exploration.
  🔵 Backtracking to (3, 5) | Battery: 100% → 95%
Exploring node (5, 6) | Terrain: flat | Battery: 95% → 90%
  ⚠️ Hazardous terrain at exploration node (5, 7). Backtracking.
  Backtracking 1 steps to safe cell (3, 4) during exploration.
  🔵 Backtracking to (3, 4) | Battery: 90% → 80%
Exploring node (6, 8) | Terrain: flat | Battery: 80% → 75%
Exploring node (7, 8) | Terrain: flat | Battery: 75% → 70%
Exploring node (8, 8) | Terrain: flat | Battery: 70% → 65%
Exploring node (8, 9) | Terrain: flat | Battery: 65% → 60%
Exploring node (9, 8) | Terrain: flat | Battery: 60% → 55%
Exploring node (10, 8) | Terrain: flat | Battery: 55% → 50%
Exploring node (11, 8) | Terrain: flat | Battery: 50% → 45%
Exploring node (11, 9) | Terrain: flat | Battery: 45% → 40%
...
Rover moved to (20, 16) | Terrain: flat | Battery: 40% → 35%
Rover moved to (20, 17) | Terrain: flat | Battery: 35% → 30%
Rover moved to (20, 18) | Terrain: flat | Battery: 30% → 25%
Detour: Battery between 20-25%. Recharge station found at (20, 21)

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



9) Contribution:

Topics	Name:
Code	Both worked together
Presentation	Vishwa
Report	Akkshat
Comparison	Both worked together

10) Help taken

We took help from various domains, firstly we looked at the documentation of the matplotlib, went through the aaima-python library, and also took help of several LLMS to debug our code. Then we searched on youtube for the related videos and went through it and then tried to combine all and come to one final code.

11) Conclusion

This project successfully demonstrated the exploration rover to find the optimal path using A* and four different heuristics - Weighted Manhattan, Euclidean distance, terrain cost weighted and battery aware. By including the cost based model, battery constraints, hazardous terrains, self charging modules, and the visualization it becomes the perfect solution to find out the optimal path from start state to the goal state. The results showed that the heuristic played a huge role in determining the optimal path. The design is made in such a way that new things can be implemented in the future and with the help of the visualization we can interpret all the important functions of the model easily and understand it correctly.